```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX 100
char stack[MAX];
int top = -1;
void push(char item) {
if (top >= MAX - 1) {
printf("Stack Overflow\n");
return;
}
stack[++top] = item;
}
char pop() {
if (top < 0) {
printf("Stack Underflow\n");
return '\0';
}
return stack[top--];
}
```

```
char peek() {
if (top < 0) {
return '\0';
}
return stack[top];
}
int isEmpty() {
return top == -1;
}
int getPrecedence(char ch) {
switch (ch) {
case '+':
case '-':
return 1;
case '*':
case '/':
return 2;
case '^':
return 3;
}
return 0;
}
```

```
int isOperator(char ch) {
return (ch == '+' || ch == '-' || ch == '*' || ch == '/' ||
ch == '^');
}
void infixToPostfix(char infix[], char postfix[]) {
int i = 0, j = 0;
char ch;
while (infix[i] != '\0') {
ch = infix[i];
if (isalnum(ch)) {
postfix[j++] = ch;
}
else if (ch == '(') {
push(ch);
}
else if (ch == ')') {
while (!isEmpty() && peek() != '(') {
postfix[j++] = pop();
}
pop();
}
else if (isOperator(ch)) {
```

```
while (!isEmpty() && getPrecedence(peek()) >=
getPrecedence(ch)) {
postfix[j++] = pop();
}
push(ch);
}
i++;
}
while (!isEmpty()) {
postfix[j++] = pop();
}
postfix[j] = '\0'; // Null terminate the string
}
void infixToPrefix(char infix[], char prefix[]) {
int len = strlen(infix);
char reversedInfix[MAX];
char postfix[MAX];
for (int i = 0; i < len; i++) {
char ch = infix[len - 1 - i];
if (ch == '(') {
reversedInfix[i] = ')';
} else if (ch == ')') {
```

```
reversedInfix[i] = '(';
} else {
reversedInfix[i] = ch;
}
}
reversedInfix[len] = '\0';
infixToPostfix(reversedInfix, postfix);
int postLen = strlen(postfix);
for (int i = 0; i < postLen; i++) {
prefix[i] = postfix[postLen - 1 - i];
}
prefix[postLen] = '\0';
}
int main() {
char infix[MAX], prefix[MAX];
printf("Enter an infix expression: ");
scanf("%s", infix);
infixToPrefix(infix, prefix);
printf("The prefix notation is: %s\n", prefix);
return 0;
}
```